

Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 3

Especificação de linguagem de programação

Introdução

- *Quais são as capacidades e limitações fundamentais dos computadores?*
 - Para isso é preciso entender: **autômatos**, **computabilidade e complexidade**
 - São áreas da **Teoria da Computação**
 - Em cada uma dessas três áreas, essa questão é interpretada diferentemente e as respostas variam conforme a interpretação

Introdução



Teoria da Computação

- Determinar o que pode e o que não pode **ser computado**,
quão **rapidamente**, com quanto de **memória** e sobre que
tipo de **modelo computacional**

Teoria da Computação

- Pode ser vista como um roteiro que nos orienta no sentido de informar o que pode e o que não pode ser efetivamente computável, explicando **porque, de que forma e com que complexidade**

Teoria da Computação

- A Teoria da Computação classifica os problemas computacionais em três classes:
 - **Problemas Indecidíveis** (ou impossíveis de serem solucionados);
 - **Problemas Intratáveis** (possíveis com recursos ilimitados, porém impossíveis com recursos limitado)
 - **Problemas Tratáveis** (possíveis de serem solucionadas com recursos limitados)

Teoria da Computação

- Esta classificação engloba problemas de toda a natureza, envolvendo desde problemas **clássicos** que fundamentam a Teoria da Computação até problemas (ou instâncias de problemas) **práticos** da Ciência da Computação, tais como:

Teoria da Computação

- *Existe programa para solucionar um determinado problema?*
- *Uma determinada solução é a melhor solução para um dado problema?*
- *Qual o significado de um determinado programa?*
- *Dado um programa qualquer, este programa está correto?*

Teoria da Computação

- A Teoria da Computação pode ser vista como um conjunto de **modelos formais** (juntamente com suas propriedades) que fundamentam a Ciência da Computação.

Teoria da Computação

- Tais modelos incluem **Autômatos** (Finitos, de Pilha e Máquinas de Turing) e **Gramáticas**, enquanto que as propriedades de interesse envolvem questões de decidibilidade, inter-relacionamento entre modelos e complexidade computacional.

Teoria da Computação

- Todos os problemas computacionais podem ser tratados (estudados) sob a ótica da **Teoria das Compiladores**.

Teoria dos Autômatos

- Lida com as definições e propriedades de modelos matemáticos de computação
- São ótimos para se começar a estudar a Teoria da Computação para depois definir **o que é computável ou não**
- Esses modelos desempenham um papel em diversas áreas aplicadas à Ciência da Computação, como por exemplo Inteligência Artificial

Teoria dos Autômatos

- **Autômatos finitos**, por exemplo, são usados em processamento de texto, compiladores e projeto de hardware
- São bons modelos para computadores com uma quantidade extremamente limitada de memória
 - Apesar da limitação é possível resolver uma grande quantidade de problemas com estas máquinas.

Teoria da Computabilidade

- Durante a primeira metade do Séc. XX, matemáticos como Alan Turing e Kurt Gödel, descobriram que certos problemas básicos não podem ser resolvidos por computadores
 - **Exemplo:** determinar se um enunciado matemático é verdadeiro ou falso (**Teoremas da Incompletude e o Problema da Parada**)



Alan Turing



Kurt Gödel

Teoria da Computabilidade

- Essa área permite observar o **poder na resolução de problemas**:
 - Problemas que **podem** ser resolvidos com algoritmos
 - Problemas que **não podem** ser resolvidos com algoritmos
- A **Máquina de Turing** é utilizada como modelo teórico universal e possui maior poder computacional dos que os computadores que conhecemos
 - Define formalmente o que é **computável ou não**

Teoria da Complexidade

- A Complexidade visa **classificar** os problemas computacionais de acordo com a sua **dificuldade computacional**
 - *O que faz alguns problemas computacionalmente difíceis ou fáceis?*
- Não está relacionada a facilidade ou dificuldade que uma pessoa tem em implementar um algoritmo

Teoria da Complexidade

- Nem todos os problemas computáveis podem ser resolvidos na práticas
 - Recursos computacionais como **tempo** e **espaço** podem ser proibitivos
- Essa teoria tem mostrado para os criptógrafos, por exemplo, o caminho dos problemas computacionalmente difíceis, em torno dos quais novos códigos revolucionários foram criados

Teoria da Complexidade

Arquitetura e hardware

O impacto da computação quântica na complexidade algorítmica

A computação quântica está silenciosamente destruindo nossa compreensão da complexidade algorítmica, expondo décadas de trabalho em otimização e arquitetura de software.

Por Alex Williams

Publicado 14 de abril de 2025

<https://cacm.acm.org/blogcacm/quantum-computings-impact-on-algorithmic-complexity/>



Especificação da linguagem de programação

- Uma linguagem de programação deve ter uma definição precisa antes de ser implementada
- No geral, essa definição consiste em duas partes:
 - **Sintaxe da linguagem**
 - **Semântica da linguagem**

Especificação da linguagem de programação

- Uma linguagem de programação pode ser definida pela descrição da aparência de seus programas (**a sintaxe da linguagem**) e do que os mesmos significam (**a semântica da linguagem**)

Especificação da linguagem de programação

- São especificadas todas as sequências de símbolos que constituem programas válidos numa dada linguagem
- São estes, e somente estes, os programas que deveriam ser aceitos e traduzidos pelo compilador
- Essa parte da definição que especifica a forma dos programas válidos é a de **sintaxe**

Especificação da linguagem de programação

- Para especificar a sintaxe de uma linguagem, apresentamos uma notação amplamente aceita, chamada de **BNF (Forma de Backus-Naur)**

Especificação da linguagem de programação

- A segunda parte da definição consiste na especificação do significado associado a cada construção da linguagem e é chamada de **semântica** da linguagem
- O programa objeto gerado pelo compilador para cada construção da linguagem está baseado nesta especificação

Especificação da linguagem de programação

- Existem **métodos formais** para especificação completa tanto da **sintaxe** como da **semântica**
- As descrições que usam os métodos atualmente existentes são muito complexas

Especificação da linguagem de programação

- Na prática
 - **Sintaxe:** métodos formais
 - **Semântica:** métodos informais

Definições indutivas

- As definições indutivas constituem o método mais comumente usado para a especificação da **sintaxe**
- Já vimos alguns exemplos em Lógica

Definições indutivas

- Suponhamos que se deseja especificar a forma de todas as expressões aritméticas que usam os símbolos de variáveis **a** e **b**, e também os símbolos de operação **+** e *****, e os **parênteses**
- Uma maneira de definir de forma geral essas expressões é por meio de regras:

Definições indutivas

1. **a** e **b** são expressões
2. se **a** e **b** são sequências de símbolos que são expressões, então a sequência de símbolos **a + b** e **a * b** são expressões
3. se **a** é uma sequência de símbolos que é uma expressão, então **(a)** é uma expressão

Definições indutivas

4. **expressões** são todas e somente as sequências de símbolos obtidas por uma aplicação das regras 1, 2 e 3, um número finito de vezes

- A regra 4 fica subentendida

Definições indutivas

- Podemos definir **expressões** como sendo o conjunto E
- Assim, E é o conjunto das sequências de símbolos que são **expressões**
- Com isso, as regras são atualizadas e ficam da seguinte forma:

Definições indutivas

1. a e b estão em E
2. se a e b estão em E , então $a + b$ e $a * b$ estão em E
3. se a está em E , então (a) está em E

Definições indutivas

- Essas definições podem ser usadas de **duas formas**:
 - Indica como **construir** sequências de símbolos que são expressões
 - Indica como **verificar** se uma dada sequência é uma expressão

Definições indutivas

- Exemplo: $(a+b)^*a$

1. $a \leftarrow$ Regra 1

Definições indutivas

- Exemplo: $(a+b)*a$

1. $a \leftarrow$

Regra 1

2. $b \leftarrow$

Regra 1

Definições indutivas

- Exemplo: $(a+b)^*a$

1. $a \leftarrow$ **Regra 1**

2. $b \leftarrow$ **Regra 1**

3. $a + b \leftarrow$ **Regra 2, linha 1 e 2**

Definições indutivas

- Exemplo: $(a+b)*a$

1. $a \leftarrow$ Regra 1

2. $b \leftarrow$ Regra 1

3. $a + b \leftarrow$ Regra 2, linha 1 e 2

4. $(a + b) \leftarrow$ Regra 3, linha 3

Definições indutivas

- Exemplo: $(a+b)*a$

1. $a \leftarrow$ Regra 1

2. $b \leftarrow$ Regra 1

3. $a + b \leftarrow$ Regra 2, linha 1 e 2

4. $(a + b) \leftarrow$ Regra 3, linha 3

5. $(a + b) * a \leftarrow$ Regra 2, linha 4 e 1

Definições indutivas

- **Exemplo:** $(a+b)*a$
 - A ordem apresentada não é a única possível, pois poderíamos trocar o **passo 1 pelo 2**, por exemplo

Definições indutivas

- Exemplo: $(a+b)*a$

1. $b \leftarrow$ Regra 1

2. $a \leftarrow$ Regra 1

3. $a + b \leftarrow$ Regra 2, linha 1 e 2

4. $(a + b) \leftarrow$ Regra 3, linha 3

5. $(a + b) * a \leftarrow$ Regra 2, linha 4 e 1

BNF

- Existe uma notação especial chamada BNF usada para definir a **sintaxe** das linguagens de programação
- **Exemplo:** $(a+b)^*a$
 - Assim, nesta notação, a definição das expressões do nosso exemplo ficaria da seguinte forma

BNF

- **Exemplo:** $(a+b)^*a$

1. $E ::= a$

2. $E ::= b$

3. $E ::= E + E$

4. $E ::= E * E$

5. $E ::= (E)$

BNF

- **Exemplo:** $(a+b)*a$

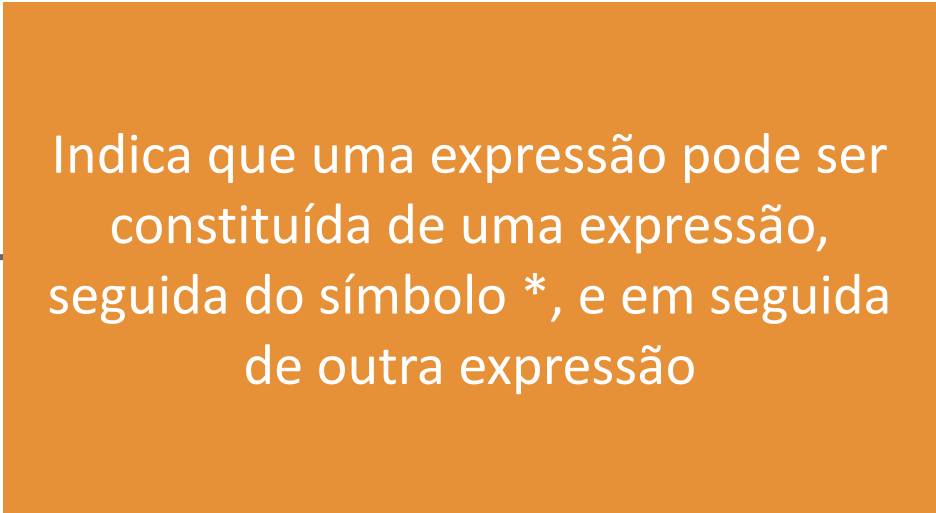
1. $E ::= a$

2. $E ::= b$

3. $E ::= E + E$

4. $E ::= E * E$

5. $E ::= (E)$



Indica que uma expressão pode ser constituída de uma expressão, seguida do símbolo $*$, e em seguida de outra expressão

BNF

- **Exemplo:** $(a+b)^*a$
 - Para esse exemplo, os símbolos **a**, **b**, **+**, *****, **(e)** são os únicos que podem entrar na formação de uma expressão
 - **E** é um símbolo auxiliar que facilita a especificação concisa das expressões
 - **::= ou** $\rightarrow$ representa “é definido como”

BNF

- É comum agruparem-se as várias alternativas da definição, separando-as com barras verticais (que significa **ou**)
- **Exemplo:** $(a+b)^*a$

$E ::= a \mid b \mid E+E \mid E^*E \mid (E)$

BNF

- A notação BNF é usada para representar uma classe de definições que em Teoria de Linguagens Formais são chamadas de **gramáticas livres de contexto**

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EOE \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EOE \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

C: Sequência de comandos
de atribuição, separados
pelo símbolo ;

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EOE \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

$b \leftarrow a + b ; a \leftarrow a * (b + a)$

Uma sequência de comandos (C) é definida como sendo um identificador (I) que pode receber uma expressão (E) ou uma sequência de comandos (C)

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EOE \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

I: Identificadores, que pode
ser a ou b

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EO E \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

O: Operadores (+ ou *)

BNF

- **Exemplo 2:** Seja a seguinte definição em BNF

1. $C ::= I \leftarrow E \mid C ; C$

2. $E ::= I \mid EOE \mid (E)$

3. $O ::= + \mid *$

4. $I ::= a \mid b$

E: Expressões, que no caso pode ser o próprio identificador, ou expressões juntamente com um operador

Elementos Matemáticos

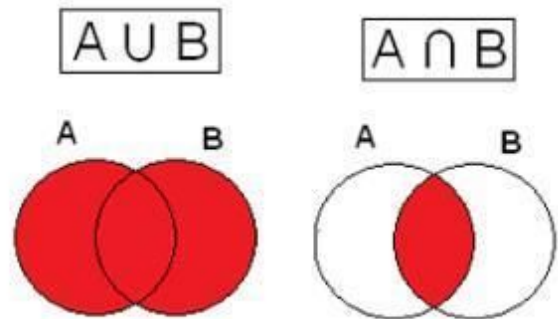
- Faremos o estudo dos objetos matemáticos básicos, ferramentas e notações que esperamos usar ao longo da disciplina:
 - **Conjuntos**
 - **Alfabeto**
 - **Palavra**
 - **Comprimento**
 - **Palavra Vazia**
 - **Prefixo e Sufixo**
 - **Subpalavra**
 - **Concatenação**
 - **Cadeia Invertida**
 - **Fechamento**
 - **Linguagem**

Conjuntos

- É um **grupo de objetos representado como uma unidade**
- Podem conter qualquer tipo de objetos, tais como, **números, letras, símbolos** e até mesmo **outros conjuntos**
- Esses objetos dentro do conjunto são chamados de **elementos** ou **membros**
 - **Conjunto vazio ($\emptyset$):** um conjunto com zero elementos

Conjuntos

Simbologia	Leitura
$\subset$	Contém
$\not\subset$	Não contém
$\in$	Pertence
$\notin$	Não pertence
$\cup$	União
$\cap$	Intersecção

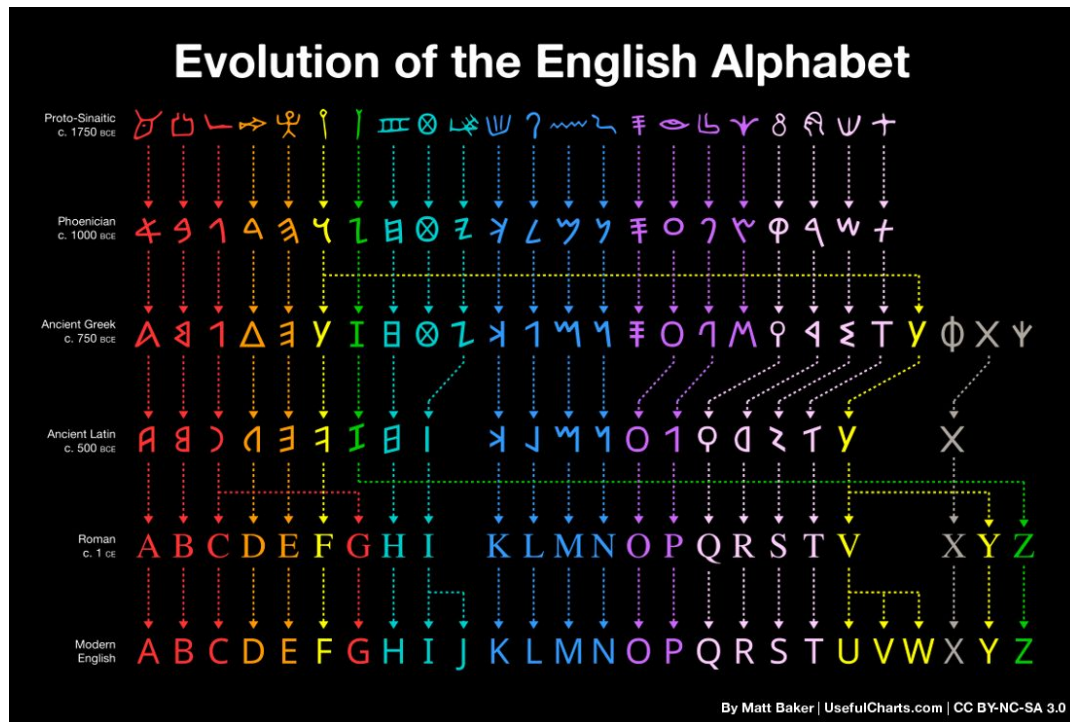


Alfabeto

- Chamaremos de **alfabeto** (ou vocabulário) a um conjunto finito e não vazio de **símbolos**
 - **Exemplos de símbolos:** letras, dígitos, caracteres especiais, ou então compostos, como **begin, if, :=**

Alfabeto

- Exemplos de alfabetos
 - ASCII
 - Grego
 - Binário



Palavra ou Cadeia

- Uma **palavra** representa uma cadeia/sequência de símbolos
 - **Exemplo:** suponha que Σ seja um alfabeto. Assim, uma **palavra** w sobre Σ é definido como:
 - $w = (w_0, w_1, w_2, \dots, w_{n-1})$ onde $w_i \in \Sigma, 0 \leq i \leq n-1$ e $n \geq 0$

Palavra ou Cadeia

- **Exemplo:** dado $\Sigma = \{0,1\}$
 - $w1 = (0,0,0)$
 - $w2 = (0,1)$
 - $w3 = (0)$
- É possível simplificar a notação de uma palavra
 - **$w = (t,e,s,t,e) = \text{teste}$**

Comprimento

- **Comprimento de palavra** é a quantidade de símbolos que uma palavra possui
 - **Exemplo:** Seja Σ um alfabeto, uma palavra $w = (w_0, w_1, w_2, \dots, w_{n-1})$, está definida sobre Σ . Assim, **o comprimento de w é n**
 - **Notação:** $|w| = n$

Comprimento

- Exemplo:
 - $\Sigma = \{a, b, c, d, e, \dots, z\}$
 - $w = (a, u, l, a)$
 - $||w|| = 4$

Palavra Vazia

- Definimos como **palavra vazia** a palavra que tem seu comprimento igual a zero
 - **Exemplo:** Seja Σ um alfabeto, seja w uma palavra que está definida sobre Σ . Se o $||w|| = 0$, então $w = \lambda$
 - **Notação:** λ ou ϵ

Atenção!
palavra vazia (ϵ) é
diferente de
conjunto vazio ($\emptyset$)

Prefixo e Sufixo

- **Prefixo** é qualquer sequência inicial de símbolos da palavra
- **Sufixo** é qualquer sequência final de símbolos da palavra
- **Exemplo:**
 - ϵ , a, ab, abc, abcd são prefixos da palavra **abcd**
 - ϵ , d, cd, bcd, abcd são sufixos da palavra **abcd**

Subpalavra ou subcadeia

- Uma **subpalavra** de uma palavra é qualquer sequência de símbolos consecutivos (contíguos) da palavra
- Portanto, **qualquer prefixo ou sufixo é uma subpalavra**
 - **Exemplo:**
 - **cad** é uma subcadeia de **abracadabra**



Concatenação

- A concatenação é a junção de palavras ou linguagens
- Exemplo:
 - Seja um alfabeto Σ , e duas palavras $w, x \in \Sigma^*$, $w = (w_0, w_1, \dots, w_{n-1})$ e $x = (x_0, x_1, \dots, x_{n-1})$. A concatenação $w.x = (w_0, w_1, \dots, w_{n-1}, x_0, x_1, \dots, x_{n-1})$
 - **$w = \text{aula}$ e $x = \text{vaga} \rightarrow w.x = \text{aulavaga}$**

Concatenação

- A palavra vazia na concatenação é um **elemento neutro**
- **Concatenação sucessiva:** representada na forma de expoente (w^n)
 - $w^2 = \text{aulaaula}$

Cadeia Invertida

- O **inverso** de uma cadeia $w = w_1 w_2 \dots w_n$ é denotado por $w^R = w_n w_{n-1} \dots w_1$
- **Exemplo:**
 - $w = \text{aula}$
 - $w^R = \text{alua}$
- $(w^R)^R = w$

Fechamento

- **Fechamento** é o conjunto de palavras possíveis dado um alfabeto
- **Exemplo:** Seja um alfabeto Σ
 - $\Sigma^n = \bigcup \Sigma^i \ (i \in \mathbb{N}) = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \cup \Sigma^n$
- **Notação:**
 - Σ^*
 - Σ^+ não contém o elemento ϵ

Fechamento

- Exemplo:

- $\Sigma = \{0,1\}$

- $\Sigma^* = \{\epsilon, \mathbf{0}, \mathbf{1}, 00, 01, 10, 11, 000, 001, \dots, 111, \dots\}$



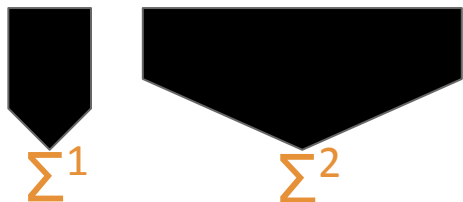
Σ^1

Fechamento

- Exemplo:

- $\Sigma = \{0,1\}$

- $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots, 111, \dots\}$



Linguagem

- Uma **linguagem (L)** ou **linguagem formal** sobre um alfabeto Σ é um conjunto de palavras ($L \subseteq \Sigma^*$)
- **Exemplo:**
 - $\Sigma = \{0, 1\}$
 - $L1 = \{\epsilon, 00, 01, 10, 11, 0000, 0001, \dots, 1111, \dots, 000000, \dots\}$
 - $L2 = \emptyset$, $L3 = \{\epsilon\}$, $L4 = \{0, 1\}^*$

Linguagem

- Uma das aplicações fundamentais e essenciais relacionadas a uma linguagem formal é a capacidade de **identificar automaticamente as palavras que fazem parte dela.**
- Um reconhecedor para uma linguagem formal L é um procedimento que ao ler qualquer palavra w indica se $w \in L$ ou se $w \notin L$.

Linguagem

- O estudo de linguagens está intimamente relacionado ao estudo das formas de representação dessas linguagens. O problema de representação de uma linguagem, por sua vez, está relacionado com o fato dela ser **finita ou infinita**

Linguagem Finita

- **Linguagem Finita** é uma linguagem que pode ser representada por enumeração.
- **Exemplo:**
 - A linguagem definida como sendo o conjunto dos inteiros positivos pares maiores que 0 e menores que 20, pode ser representado por: **$L = \{2, 4, 6, 8, 10, 12, 14, 16, 18\}$**

Linguagem Infinita

- **Linguagem Infinita** é a linguagem que impossibilita de usarmos enumeração. É preciso utilizar uma representação finita.
- **Exemplo:**
 - A linguagem definida como sendo o conjunto dos inteiros pares poderia ser representada por: $L = \{2, 4, 6, 8, 10, \dots\}$